Finsite

the finance site, for financial insight

DevHex

Technical Architecture & Design Review

A deep dive into the design and engineering behind the finance tracker

DevHex

High-Level System Overview

Core Architecture

Built as a **Single Page Application (SPA)** on a strict Model-View-Controller (MVC) foundation. By building it using native web standards we avoid framework bloat and complexity. Key components include interchangeable page views driven by the view layer and an event pipeline for user actions and the programs reaction.

Storage & Visualization

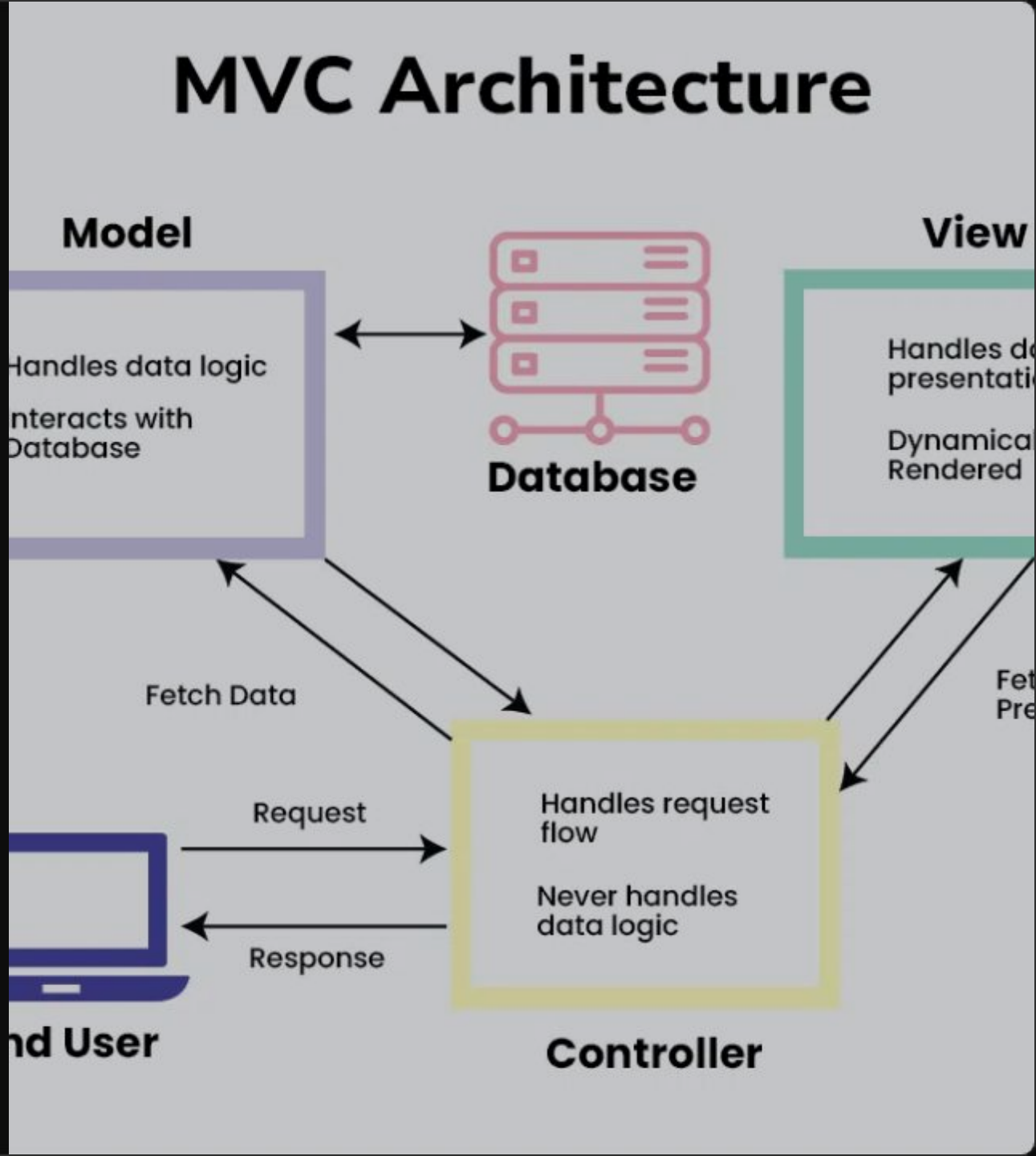
Data is persisted via a **Promise-based IndexedDB** abstraction, its fully local to protect client side privacy. Visuals are handled by a custom **Charting system** that decouples data aggregation from rendering libraries.

MVC Architecture

A strict unidirectional control flow ensures predictability:

- **Model:** Domain logic, state, and storage IO.
- **View:** DOM manipulation and Event trapping.
- **Controller:** Policy layer binding User Intent to Model updates.

The Controller effectively acts as the "Traffic Cop" for the application.



Core Architecture

finsiteModel.js

the Model owns all domain state and business rules in memory, acting as the single source of truth for transactions, groups, and categories. The model exposes high level operations (add/delete/import, grouping, category logic) and returns already aggregated domain results, but knows nothing about how they're persisted or displayed.

Responsibilities

- Load/persist transactions, groups, and categories via storage service
- Maintain in-memory copies of all data (this.data.transactions, this.data.groups, this.data.categories)
- Incrementally update aggregate Maps (_timeBuckets, _groupTotals, _cachedMetrics) on every add/delete
- Provide O(1) dashboard summaries using pre-computed aggregates
- Validate and normalize transaction inputs before persistence

Core Architecture

finsiteView.js

The View manages the SPA shell and swaps self contained web components (Dashboard, Transactions, Categories) into the main content area, wiring data into them via setters and listening for their events. It utilizes helpers such as categoryAggregator.js, formatters.js, icons.js, and debugService.js to keep the logic cohesive and reusable.

Responsibilities

- Render the two-pane layout (sidebar + main content area)
- Navigate between pages (dashboard, transactions, categories) by swapping components
- Set up event listeners on container to catch bubbled events from components
- Forward component events (add-transaction, request-delete-group) to Controller via handlers
- Push model data to active components via setTransactions(), setTaxonomy(), setData()
- Wire model reference to categories and transactions components
- Update dashboard charts and panel with pre-aggregated data from Model
- Provide lifecycle callbacks (onTransactionAdded, onGroupDeleted) for Controller to notify components

Core Architecture

finsiteController.js

the Controller sits above the model as an orchestration layer that translates user intents into model calls and then feeds updated data into the UI. It handles user actions (add transaction, delete group, navigate), orchestrates data flow by calling Model methods and passing results to View, and manages dashboard refresh logic with optimized aggregation queries (light vs heavy update)

Responsibilities

- Initialize application by loading Model data and rendering View
- Handle user actions forwarded from View (add transaction, navigate, delete group)
- Call Model methods to persist changes and retrieve data
- Call View methods to update UI with fresh data
- Decide when to refresh dashboard (after data changes, on navigation)
- Choose between light updates (with animation) vs heavy updates (bulk import, no animation)

Storage Service

storageService.js is a self contained persistence module built on top of IndexedDB and exposed through a small, Promise-based service API, so none of the core architecture ever talks to the database directly.

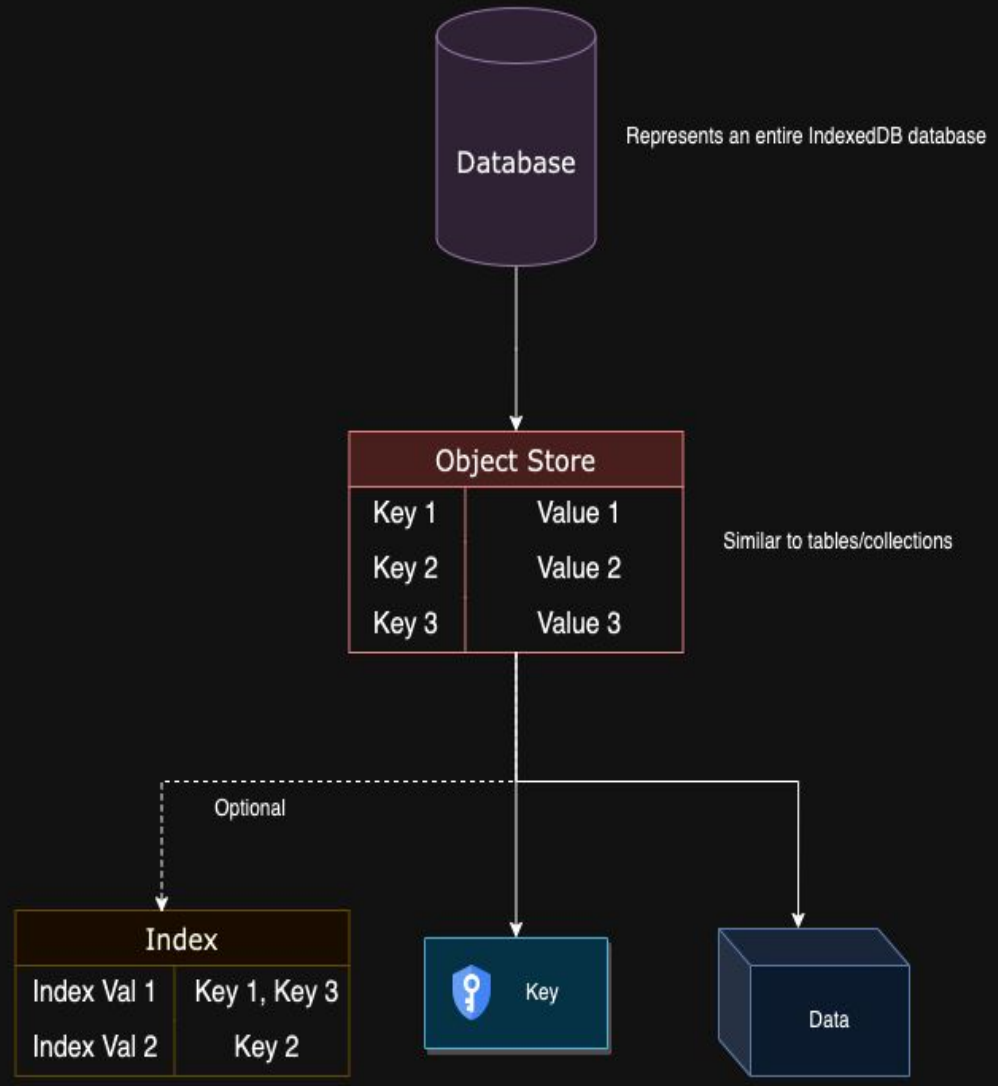
It manages object stores for:

transactions

groups

categories

Because it is isolated, it can be swapped out to a remote API or different database with minimal impact on the rest of the system



Storage Service

storageService.js

The storage service provides a clean Promise based API wrapping IndexedDB for all CRUD operations on transactions, groups, and categories. It handles database initialization, schema versioning, error handling, and transaction atomicity, abstracting IndexedDB's event based API from the model.

Responsibilities

- Initialize and manage IndexedDB database connection (finsiteDB)
- Create and upgrade database schema (object stores and indexes)
- Provide async CRUD operations for transactions (add, getAll, delete, clear)
- Provide async CRUD operations for groups (add, getAll, delete)
- Provide async CRUD operations for categories (add, getAll, batch update)
- Normalize and validate IDs before database operations
- Handle IndexedDB transaction management and error wrapping
- Ensure batch operations are atomic (all succeed or all fail)

Charting System

chart-core.js acts as the factory layer for Chart.js.

It lazy-loads the library and standardizes configurations (fonts, tooltips, colors) globally.

Benefits:

- Consistent styling across all charts.
- Chart components only need to pass data, not config.
- Performance toggles for animation during bulk operations.

Charting System

chart-core.js

The chart core centralizes Chart.js initialization and configuration module. Lazy-loads Chart.js on first use (avoiding initial bundle bloat), applies global defaults, provides factory functions for creating line and bar charts, and ensures all charts use consistent styling and behavior.

Responsibilities

- Lazy-load Chart.js via dynamic script injection when first chart is rendered
- Register Chart.js components (scales, tooltips, etc.) with minimal surface area
- Apply global Chart.js defaults (fonts, colors, animations, tooltips)
- Provide factory functions (createLineChartConfig, createBarChartConfig) for consistent chart creation
- Export `formatCurrency()` helper for axis labels and tooltips
- Expose CHART_COLORS palette for multi-category visualizations
- Cache Chart.js instance to prevent duplicate loads

Charting System

spending-chart.js

Dashboard chart container component that renders two Chart.js visualizations (line chart for 6-month spending trend, bar chart for top groups breakdown) plus KPI metric cards. Receives pre-aggregated data from Model via View and handles chart updates efficiently with animation toggles for bulk operations.

Responsibilities

- Render metrics row with 4 KPI cards (this month, % change, last month, 6-month avg)
- Initialize and manage two Chart.js instances (line chart, bar chart)
- Update charts when new data provided via updateChartData() without recreating instances
- Disable animations for "heavy updates" (bulk imports) to improve performance
- Lazy-load Chart.js via chart-core module on first render
- Destroy Chart.js instances on component removal to prevent memory leaks
- Format currency values for metrics display

Charting System

category-chart.js

Reusable bar chart card component for the Categories page. Displays spending breakdown for a single group (Household, Wealth, Expenses, custom groups) as a vertical bar chart with subcategories as bars. Clickable to open modal with transaction details. Includes delete button for custom groups.

Responsibilities

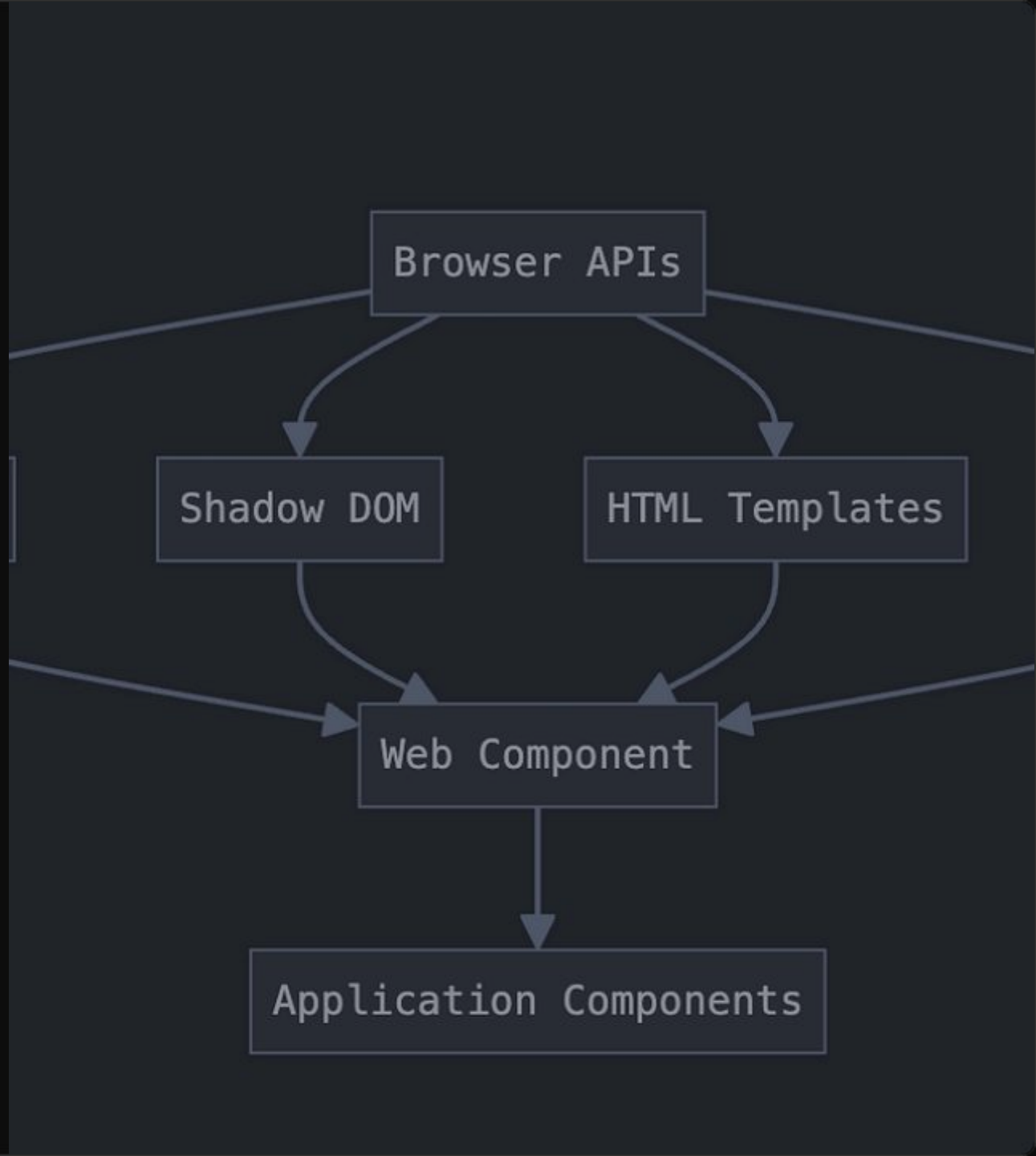
- Render group summary card with icon, name, total spent, transaction/category counts
- Initialize Chart.js bar chart showing category spending for one group
- Handle click events to open group detail modal
- Show delete button for custom groups (isCustom flag)
- Emit group-selected event when card clicked
- Emit request-delete-group event when delete button clicked
- Destroy Chart.js instance on component removal to prevent memory leaks

Web Component Architecture

The UI is built from modular web components. Each component encapsulates its own styles and template logic.

Communication Protocol:

- ↓ **Props Down:** Data is passed via setters (e.g., el.transactions = [...]).
- ↑ **Events Up:** Actions are emitted as Custom Events (e.g., addTransaction).



View Components

dashboard.js

Displays the main dashboard overview with stat cards (total spent, weekly transactions, monthly spending), delegated charts (line chart for 6-month trend, bar chart for top 5 groups), and recent activity list.

sidebar.js

Persistent vertical navigation sidebar with collapsible icon-only mode. Provides primary navigation , theme toggle, and visual branding. coordinates navigation events for parent pages.

transactions.js

Full-featured transaction management page with date-grouped list, advanced filtering, multi-sort, and manual transaction entry modal. Provides complete

categories.js

Displays spending breakdown by custom and default groups on the Categories page. Renders multiple charts (one per group) and provides models for viewing group details and creating new custom groups. Allows users to delete custom groups with confirmation.

Utility Modules: Separating Concerns

categoryAggregator.js

Pure functional utilities for computing category spending totals, group breakdowns, and transaction filtering. Enables both Model and Components to build aggregate views without duplicating calculation logic.

formatters.js

Transforms raw model data into human-readable display formats. Maintains MVC separation by keeping display logic out of Model layer, enabling Model to be used in non-UI contexts and simplifying localization/theming..

Utility Modules: Centralization

debugService.js

Centralized debug logging control for the entire MVC stack. Provides single toggle to enable/disable diagnostic logging across Model, View, Controller, and Components, avoiding scattered debug flags and console.log statements.

icons.js

Single source of truth for all emoji icons used throughout the application. Maps category/group IDs to emoji characters and provides picker options for custom group creation, ensuring visual consistency across components.

Testing & Linting

- **Vitest for unit & integration testing**
- **Playwright for E2E**

Linter: we used ESLinter and used AirBnBs template

Rules we disabled:

- no-console - disabled to allow console debugging
- no-plusplus - disabled because i++/i-- are part of our styling
- no-underscore-dangle - disabled because our code uses _method/_field to indicate internal methods and vars
- import/extensions - disabled because native browser ES modules require explicit .js extensions
- import/prefer-default-export - disabled because our code uses named exports.
- class-methods-use-this - disabled because component helper methods can live as instance methods
- max-len - disabled because our strings/config objects are clearer unbroke

Data Flow: "Add Transaction"

1. Component
User submits form.
finsite-transactions
emits event.

2. Controller
Catches event.
Calls model.add().
Handles errors.

3. Model
Persists to DB.
Updates Aggregates.
Returns new State.

4. View
Receives new data.
Updates DOM.
Refreshes Charts.

Coding Principles

- **Single Source of Truth (SSOT):** The Model holds the definitive state; DOM and Storage are reflections.
- **Separation of Concerns (SRP):** Logic (Model), Orchestration (Controller), and UI (View) never bleed boundaries.
- **RAIL Performance Model:** Optimized for Response, Animation, Idle, and Load. O(1) dashboard queries.
- **Conscious Coupling:** Modules interact through explicit public APIs only.

What Went Wrong

- ❌ **Horizontal instead of vertical progression:** The project was built section by section rather than completing the base layer and adding to it.
- ❌ **Structural concept was not unified:** Each person was tasked with building a section, however, each section followed a different structure
- ❌ **AI Frankenstein:** Since each section was built according to a different structure the use of AI turned to abuse to mutate every section to confine to the structure detailed in this presentation
- ❌ **Almost performative:** Some essential parts of the development process were neglected and only completed to meet the checklist, some later than others
  - Linter
  - Miro
  - testing (crazy i know)
  - CI/CD

Q&A

Thank you